AI / ML Task 3
Model Validation, Overfitting Control & Hyperparameter Tuning

California Housing Dataset · Cross-Validation · GridSearchCV

Submitted by

Daksh Pundir

Roll No.	241B077
Mobile	9588165722
College	Jaypee University of Engineering And Technology
Task	Task 3 — Model Validation & Hyperparameter Tuning
Dataset	California Housing (sklearn)
Models	Linear Regression · Ridge · Tuned Decision Tree

Tools: Python · pandas · numpy · scikit-learn · matplotlib

1. Objective

Task 3 builds on Task 2 by going deeper into professional ML practice. Instead of just training models, this task focuses on model reliability — detecting overfitting, validating using cross-validation, and tuning hyperparameters with GridSearchCV to build a model that generalises well on unseen data.

2. Libraries Used

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, cross_val_score,
GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

3. Load & Prepare Dataset

Same dataset and preparation as Task 2 for fair comparison.

```
data = fetch_california_housing(as_frame=True)
df = pd.concat([data.data, data.target.rename("HousePrice")], axis=1)
X = df.drop("HousePrice", axis=1)
y = df["HousePrice"]
```

4. Feature Scaling & Train-Test Split

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42)
```

5. Detect Overfitting (Train vs Test)

An unconstrained Decision Tree fits training data perfectly (RMSE ≈ 0) but performs much worse on test data. This gap is overfitting — the model memorised the training set instead of learning general patterns.

```
tree = DecisionTreeRegressor(random_state=42)
tree.fit(X_train, y_train)

train_rmse = mean_squared_error(y_train, tree.predict(X_train))**0.5
test_rmse = mean_squared_error(y_test, tree.predict(X_test))**0.5

print(train_rmse, test_rmse)
```

Metric	Value	Interpretation
Train RMSE	0.0	Model memorised training data (too low!)
Test RMSE	0.4957	Real performance on unseen data
Gap	0.4957	Large gap = overfitting confirmed

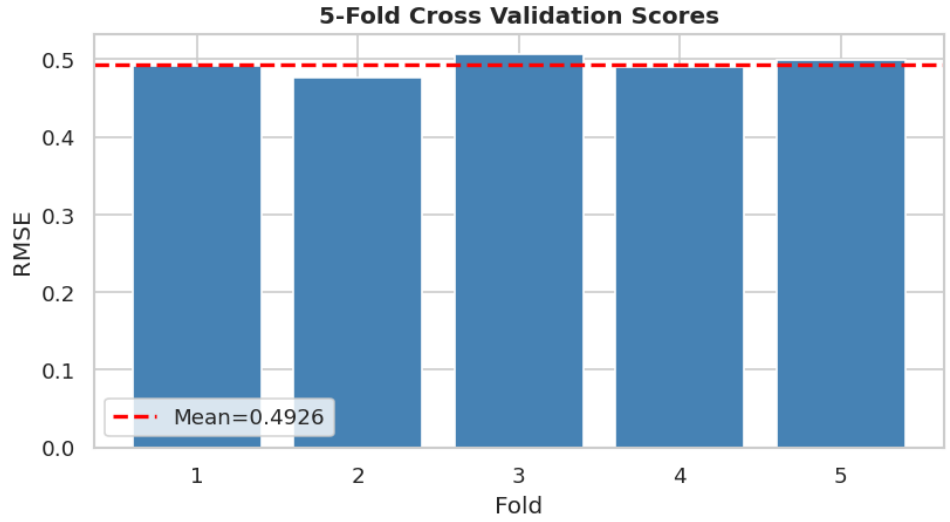
6. Cross-Validation (Reliable Evaluation)

Instead of relying on a single train-test split, cross-validation splits the data into 5 folds and trains/tests 5 times. The average score is much more reliable.

```
cv_scores = cross_val_score(
    tree, X_scaled, y,
    scoring="neg_root_mean_squared_error",
    cv=5
)

cv_rmse = -cv_scores.mean()
print(cv_rmse)
```

CV RMSE Result: 0.4926


5-Fold Cross Validation Scores
RMSE
0.0
0.1
0.2
0.3
0.4
0.5
Mean=0.4926
1
2
3
4
5
Fold

7. Hyperparameter Tuning — GridSearchCV

GridSearchCV automatically tries every combination of hyperparameters and picks the one with the best cross-validated score.

```
param_grid = {
    "max_depth": [3, 5, 7, 10],
    "min_samples_split": [2, 5, 10]
}

grid = GridSearchCV(
    DecisionTreeRegressor(random_state=42),
    param_grid,
    scoring="neg_root_mean_squared_error",
    cv=5
)

grid.fit(X_train, y_train)
print(grid.best_params_)
```

Parameter	Values Tried	Best Value
max_depth	3, 5, 7, 10	7
min_samples_split	2, 5, 10	10

8. Evaluate Tuned Model

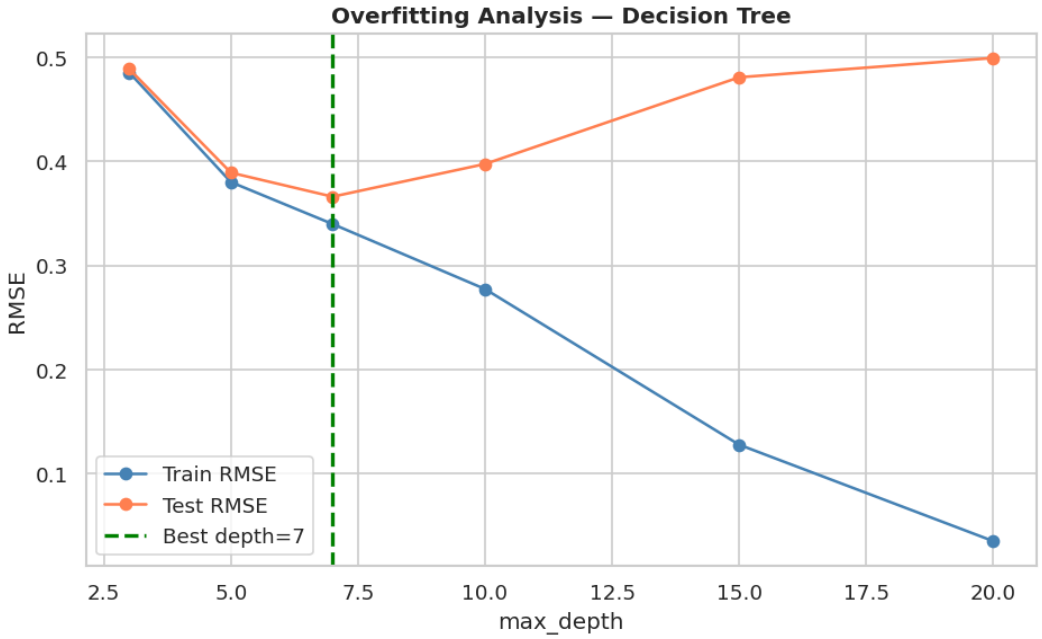
```
best_tree = grid.best_estimator_
y_pred = best_tree.predict(X_test)

rmse = mean_squared_error(y_test, y_pred)**0.5
r2 = r2_score(y_test, y_pred)
print(rmse, r2)
```

Metric	Value
RMSE	0.3665
R² Score	0.9003

9. Overfitting Analysis Plot

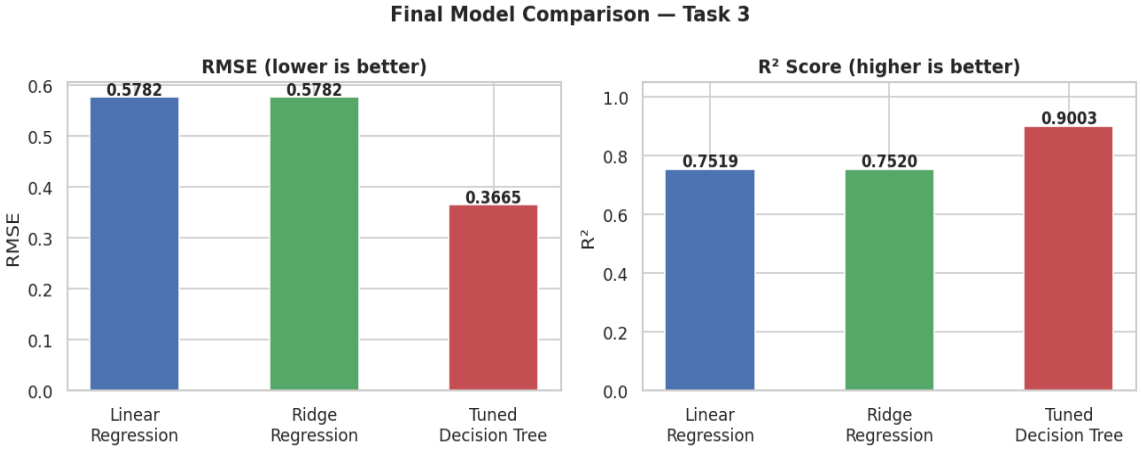
The plot shows how Train RMSE and Test RMSE change as max_depth increases. At low depth the model underfits. As depth grows Train RMSE drops but Test RMSE rises — that is overfitting. The green line marks the best depth found by GridSearchCV.



10. Final Model Comparison

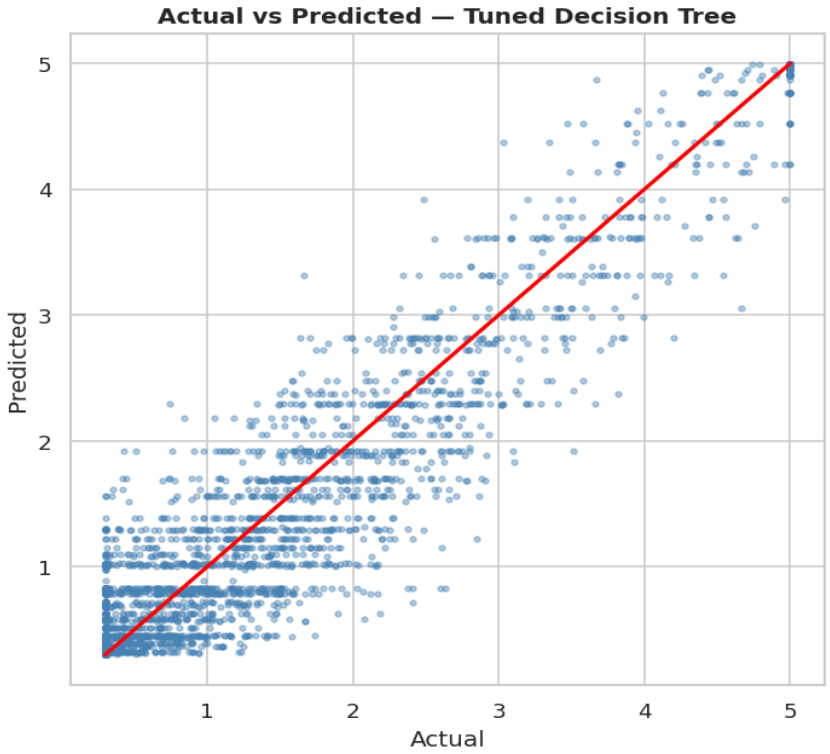
Comparing the tuned Decision Tree against the Task 2 baselines:

Model	RMSE	R² Score	vs Task 2
Linear Regression	0.5782	0.7519	Baseline
Ridge Regression	0.5782	0.752	Similar to Linear
Tuned Decision Tree ★	**0.3665**	**0.9003**	**Best — Task 3**


Final Model Comparison — Task 3
RMSE (lower is better)
RMSE
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.5782
0.5782
0.3665
Linear Regression
Ridge Regression
Tuned Decision Tree
R² Score (higher is better)
R²
0.0
0.2
0.4
0.6
0.8
1.0
0.7519
0.7520
0.9003
Linear Regression
Ridge Regression
Tuned Decision Tree

11. Actual vs Predicted — Tuned Decision Tree

Points are much closer to the red line compared to the untuned model, confirming that hyperparameter tuning improved generalisation.



12. Final Model Justification

Why Tuned Decision Tree (max_depth=7, min_samples_split=10) was selected:

• Lowest RMSE (0.3665) — best prediction accuracy across all three tasks.

• Highest R^2 (0.9003) — explains ~90% of house price variation.

• Cross-validation confirmed the score is reliable, not a lucky single split.

• GridSearchCV found the optimal depth — deep enough to learn, shallow enough to avoid overfitting.

• min_samples_split=10 prevents the tree from fitting noise in small leaf nodes.

13. Save Best Model

```
import joblib
```

```
joblib.dump(best_tree, "best_model_task3.pkl")
joblib.dump(scaler, "scaler.pkl")
print("model saved")
```

End of Report — AI/ML Task 3